
Does Graph Geometry Help Forecasting the S&P 100?

Antonio Lorenzo Díaz Meco and Javier Ríos Montes
Final Project — Geometric Deep Learning
GitHub Repository

Abstract

This final project explores the potential benefits of incorporating graph geometry into stock forecasting models, specifically focusing on the S&P 100 index. We implement two baseline models, a Random Forest and an LSTM, and compare their performance against a Graph Neural Network (GNN) that captures the relationships between stocks using a graph structure. Beyond evaluating the forecasting performance of the models, we aim to analyze whether graph symmetries might or not be the optimal inductive bias to include in stock forecasting models. Forcing a model to be invariant to certain symmetries may be beneficial in some cases, but it may also limit the model's ability to capture important relationships in the data and, as we hypothesize, incentivize the model to learn spurious relationships that aren't actually present in the data.

1 Introduction

Stock forecasting is a challenging task that, due to its economic implications, has been an attractive area of research for decades. In specific, AI and machine learning techniques have been widely applied to this problem, with varying degrees of success. Despite the advances in the matter, many current approaches are limited to using technical analysis to capture historical trends of each stock price. In this paper, we explore the potential benefits of incorporating graph geometry into stock forecasting models, specifically focusing on the S&P 100 index. We decide to focus on this subset of the stock market to reduce the complexity of the problem and to be able to focus on the relationships between the stocks, which is where we expect graph geometry to provide the most value.

In indeces such as the S&P 100, there are important relationships between individual stocks, such as shared market sectors or financial indicators, that may not be fully captured by traditional forecasting models and that could be better modeled using graph-based approaches. By representing the stocks as nodes in a graph and their relationships as edges, we can leverage geometric deep learning techniques to capture these interactions and, potentially, improve forecasting performance.

To evaluate this idea, we implement two baseline models, a Random Forest and an LSTM, and compare them against a Graph Neural Network (GNN). These baselines are chosen because they are widely used in time-series forecasting and capture different aspects of the problem: the Random Forest exploits non-linear feature interactions, while the LSTM models temporal dependencies. We frame the task as a node classification problem, where the goal is to predict whether each stock will move up, remain neutral, or move down over a given time window. We evaluate the models using accuracy, precision, recall, and F1-score, and analyze how the inclusion of graph structure affects forecasting performance.

2 Related Work

Financial forecasting surged as a research area in the late 19th century, with the introduction of statistical methods (such as those developed by Irving Fisher) to analyze market trends. It wasn't, though, until the mid-20th century that advances in computing power allowed for the application of more complex models. All in all, we can roughly divide the evolution of financial forecasting into three main eras: the pre-computing era, the statistical era, and the machine learning era. The pre-computing era relied on simple heuristics and technical analysis.

The statistical era saw the introduction of parametric models that, while being more expressive than heuristics, were based upon explicit assumptions about the underlying data distribution [3]. For example, ARIMA models the conditional mean of the time series as a linear function of past observations and errors, while GARCH models the conditional variance as a function of past squared errors and variances.

The machine learning era, currently under process, is characterized by the use of non-parametric models that can capture complex relationships in the data without making strong assumptions about the underlying distribution. The last decades have seen a surge in implementations of machine learning models for financial forecasting, including general-purpose models such as Random Forests [4] and time-series specific models such as LSTMs [10] and Gated Recurrent Units (GRUs) [1]. With the introduction of the Transformer architecture [11], attention-based architectures surged as a promising alternative to recurrent models, as they allow for long-range dependencies to be captured more effectively and in a more parallelizable way. In the specific context of stock forecasting, attention-based models have been shown to outperform recurrent models in certain settings [5].

Last, there has been a growing interest in the use of graph-based models for financial forecasting. Lead by the assumption that the relationships between stocks can provide valuable information for forecasting (and may not be fully captured by traditional models, as they lack the necessary inductive bias to model these relationships), the last years have witnessed a boom in the implementation of GNN-based models for stock forecasting. These range from purely graph-based models [8] to hybrid models that combine graph-based and sequence-based architectures [7, 9]. Added to this, the hybrid paradigm has evolved from simple combinations of sequence-based and graph-based models to more complex architectures in which the graph structure is integrated into the sequence-based model itself, such as TCGNs (Temporal Convolutional Graph Networks) [12].

In general, the results in the literature suggest that incorporating graph geometry into stock forecasting models can lead to improved performance, especially when the relationships between stocks are strong and informative. This does come with a caveat, though, as the performance of graph-based models can be sensitive to the choice of graph structure and the quality of the data used to construct the graph (more so than with simpler models as LSTMs or Random Forests). In this sense, we hypothesize that, due to the variety in graphs we may construct on a given index (for example, based on different financial indicators or with different thresholds for edge creation), the performance of the GNN may vary significantly depending on the graph structure used. Thus, the results of this project will additionally contribute to the understanding of the impact of graph structure on forecasting performance, which is an important aspect to consider when applying graph-based models to financial forecasting.

3 Background

Throughout the project, we will be using the following models:

- **Random Forest:** an ensemble learning method that constructs a multitude of decision trees during training and outputs the majority class as the prediction. Given a sufficiently informative feature set, Random Forests can capture complex relationships in the data and achieve competitive performance even compared to more complex models.
- **LSTM (Long Short-Term Memory):** a type of recurrent neural network that is particularly well-suited for sequential data (and handles long-range dependencies better than its predecessors, such as vanilla RNNs).

- **GNN (Graph Neural Network)**: a type of neural network designed to operate on graph-structured data. In our case, we will be using a specific architecture that combines an LSTM to capture the temporal evolution of each stock with a GAT (Graph Attention Network)

Focusing on the mathematics behind the GNN, we can describe GAT as a particular case of the more general message-passing framework for GNNs. In this framework, the representation of each node is updated by aggregating information from its neighbors. Specifically, in a GAT layer, the representation of a node i at layer $l + 1$ is computed as follows:

$$h_u^{(l+1)} = \sigma \left(\sum_{v \in \mathcal{N}(u)} \alpha_{uv}^{(l)} W^{(l)} h_v^{(l)} \right) \quad (1)$$

where $h_u^{(l)}$ is the representation of node u at layer l , $\mathcal{N}(u)$ is the set of neighbors of node u , $W^{(l)}$ is a learnable weight matrix, σ is a non-linear activation function (ELU in our case), and $\alpha_{uv}^{(l)}$ is the attention coefficient that determines the importance of node v 's features to node u at layer l (learned during training). Thus, it can also be interpreted as a generalization of the Graph Convolutional Network (GCN) layer, with the information from the neighbors being weighted by the attention coefficients instead of being averaged.

These weights are computed, by means of a Neural Network, as:

$$\alpha_{uv}^{(l)} = \frac{\exp \left(\text{LeakyReLU} \left(a^T [W^{(l)} h_u^{(l)} \parallel W^{(l)} h_v^{(l)}] \right) \right)}{\sum_{k \in \mathcal{N}(u)} \exp \left(\text{LeakyReLU} \left(a^T [W^{(l)} h_u^{(l)} \parallel W^{(l)} h_k^{(l)}] \right) \right)} \quad (2)$$

where a is a learnable weight vector and $\parallel$ denotes concatenation. This allows the model to learn different weights for different neighbors, which can be particularly useful in our case, as some relationships between stocks may be more informative than others for forecasting.

The computation of the hidden state $h_u^{(l+1)}$ for each node u in the GAT layer is performed in parallel for each head (so that different heads may learn different aspects of the relationships between stocks) and then concatenated to form the final representation for the next layer.

4 Method

4.1 Experimental Setup

The data used in this project consists of the historical stock prices of the companies included in the S&P 100 index, along with relevant financial indicators, depending on the specific model. We use a set of 10 features for each stock (excluding Ticker, Date and the Class label). The complete list of features can be found in the Appendix A, but, in general, they can be categorized into three groups:

- **Price-based features**: such as the adjusted closing price, daily log return, and future weekly log return (used to define the class labels).
- **Volume-based features**: such as the daily trading volume and the normalized trading volume.
- **Technical indicators**: such as moving averages, relative strength index (RSI), and rolling volatility.

In addition to this, for the LSTM model, we include the sequence of the past 20 days of the above features as input, while for the GNN model, we construct two different graph structures based on the following relationships between stocks. The following is a summary of these relationships, but a deeper description of the computation of the graph structure can be found in the Appendix B:

- A correlation graph, where each stock is connected to its top- k most correlated neighbors using training-set stock returns, with additional sector-similarity edges.
- A distributional-similarity graph, where each stock is connected to its top- k closest neighbors according to the Jensen-Shannon divergence between return distributions, with additional sector-similarity edges.

We construct these two **static** graph structures using only the training data, to avoid temporal leakage.

All data was obtained from Yahoo Finance using the `yfinance` Python library, for the period between January 4, 2021 and December 31, 2025. The choice of this period is motivated by two main reasons: (i) 5 years is a common time frame used in the literature for stock forecasting, as it provides a good balance between the amount of data available for training and the relevance of the data (as older data may not be as relevant for forecasting future stock movements), and (ii) it avoids, at least theoretically, the impact of the COVID-19 pandemic on the stock market, which introduced a significant amount of noise and structural changes in the market that could have affected the performance of the models ¹.

4.2 Data and Preprocessing

In order to obtain the information that we are to feed into our models, we followed the following steps:

- We obtained the list of stocks in the S&P 100 index, along with their corresponding ticker symbols. In specific, we filtered this list to the stocks that have been in the index for the entire period of time considered (January 4, 2021 to December 31, 2025), which resulted in a total of 88 stocks. Even though this filtering step reduces the number of stocks we are considering, we consider the reduction small enough to not significantly affect the performance of the models, while it allows us to avoid dealing with structural changes in the index.
- For each stock, we obtained the data mentioned in the previous section, computing the necessary features. As to the labels, we decided to define a threshold ϵ on the future Weekly Log Return to determine the class labels, in order to avoid labeling small fluctuations as significant movements. Specifically, we defined the class labels as follows:
 - Up: if the future Weekly Log Return is greater than ϵ
 - Neutral: if the future Weekly Log Return is between $-\epsilon$ and ϵ
 - Down: if the future Weekly Log Return is less than $-\epsilon$

We set ϵ to 0.01, which corresponds to a 1% change in the stock price over a week. Though this threshold is rather arbitrary, it yields a good balance between the number of samples in each class, as mentioned in C.2.

- We split the data into training, validation, and test sets, using a chronological split to avoid data leakage. Specifically, training data ends on December 31, 2023, validation data ends on December 31, 2024, and the remaining observations are used for testing. This divides the dataset into roughly 3, 1, and 1 years, respectively, and makes the splits comparable across models.

This is done in different ways for the different models, though. For the Random Forest, we assign the split in the tabular dataset according to the date of each sample. For the LSTM, we create sequences of 20 days for each stock and assign their split according to the reference date of each sequence. For the GNN, we create the graph structure using the data from the training set, and then split the snapshots of the graph (this is, every node and edge at a given time step).

These preprocessing steps are implemented as a reproducible pipeline that can be executed from the repository root with `uv run python -m src.data.run_pipeline`.

4.3 Models and Hyperparameters

The GNN experiments are trained using AdamW with weight decay, cross-entropy loss, gradient clipping, a fixed random seed, and early stopping based on validation macro-F1. The best checkpoint according to validation macro-F1 is saved during training and reloaded before computing the final test score.

The architectures of our models are as follows:

¹After careful analysis, we assumed that the turmoil caused by the COVID-19 pandemic in the stock market had a significant impact until the end of 2020, and that, from January 2021 onwards, the market had stabilized enough for the data to be considered as not significantly affected by the pandemic.

- **Random Forest:** we use a Random Forest with 200 trees, a maximum depth of 12, and a minimum of 50 samples per split and 20 per leaf. We add class-weighting to handle the slight class imbalance in the dataset.
- **LSTM:** we use a 3-layer bidirectional LSTM with a hidden size of 128, followed by two-layer MLP with hidden sizes of 64 and 32, respectively, ReLU activations and dropout of 0.2. We train it with AdamW with a learning rate of $5e-4$, weight decay of $1e-3$, class-weighted cross-entropy loss, and early stopping with a patience of 15 epochs.
- **GNN:** Our GNN architecture combines a temporal encoder with a graph attention network. Each node corresponds to one stock and carries a 20-day feature window. We first encode this sequence with a 2-layer bidirectional LSTM (hidden size 128) and apply a learned temporal attention mechanism to aggregate the per-day information into a single node embedding. We include a visualization of the architecture in Figure 1.

On top of these embeddings, we stack two GATConv layers with edge features: the first has 4 attention heads and a hidden size of 64 (per head), the second aggregates them into a single 64-dimensional representation. We include Batch Normalization (to deal with the inherent non-stationarity of the data), ELU activations, residual connections (to avoid over-smoothing and vanishing gradients), and dropout of 0.3.

Finally, we apply a 2-layer MLP with hidden sizes of 64 and 32, respectively, ReLU activations and dropout of 0.3, to obtain the final class probabilities. We train the model with AdamW with a learning rate of $6e-4$, weight decay of $1e-4$, class-weighted cross-entropy loss, gradient clipping with a max norm of 1.0, and early stopping with a patience of 40 epochs (to allow the model to converge given the complexity of the architecture and the noisiness of the data).

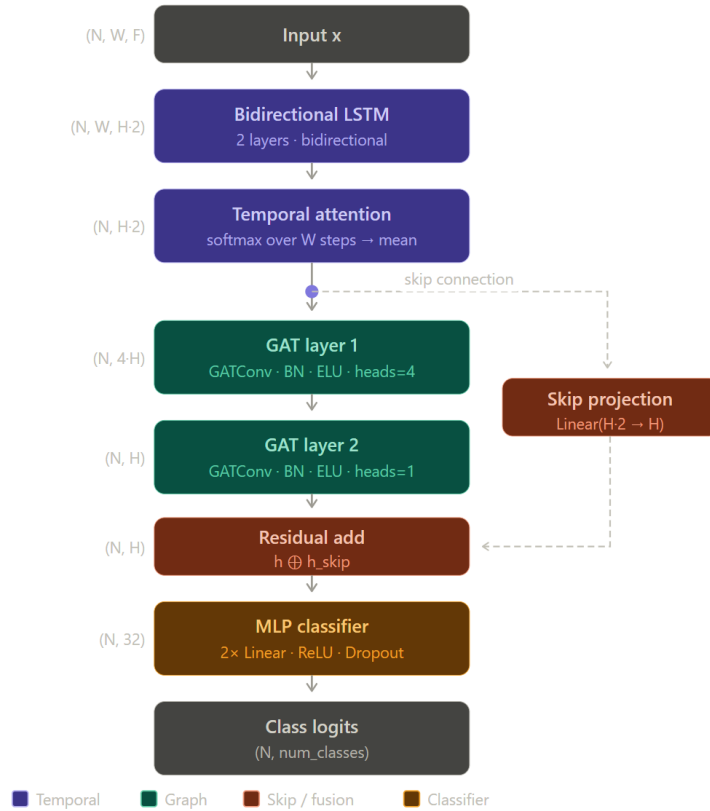


Figure 1: Illustration of the GNN architecture we implement (courtesy of Claude). The temporal encoder (LSTM + attention) is applied independently to each node, while the GAT layers capture the relationships between stocks. The final MLP layer is applied independently to each node to obtain the class probabilities.

5 Results and Discussion

5.1 Quantitative Results

Training and evaluation of the models is implemented in a modular way that can be executed from the repository root with `uv run python -m src.run_experiments`, and the main hyperparameters can be configured in `config.py`. We will first present the Training curves for the three variants 2. We don't include the Random Forest curves as it is fitted in a single step and doesn't follow the same training procedure as the other models. After this, we will present the final test scores for all models and discuss the results.

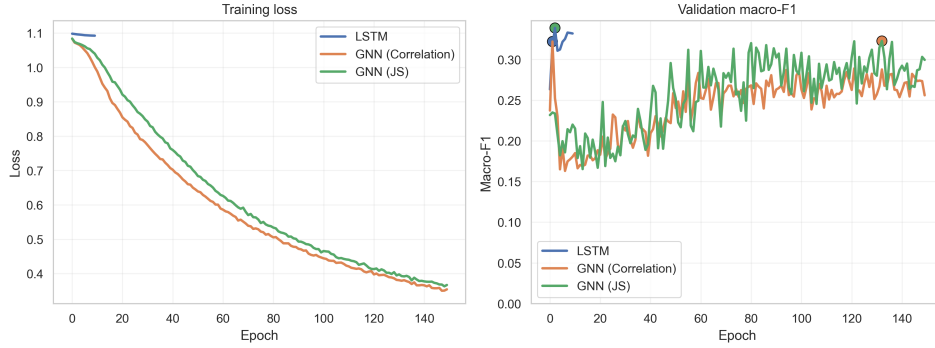


Figure 2: Training curves for the three variants. We can observe that the LSTM was cut short at the 10-epoch mark, since it didn't show any improvement in the validation macro-F1. On the other hand, both GNN variants improved mainly in the loss, while the validation macro-F1 remained rather stagnant. The JS-based GNN shows a slightly better performance than the correlation-based one. Best checkpoints are highlighted with a circle for each model.

From the training curves, we can observe several things:

- The LSTM doesn't perform well, and isn't expressive enough (our hypothesis) to capture the relationships between stocks, so training doesn't yield any improvement in the validation macro-F1, and it is cut short early on.
- Both GNN variants seem to have their performance bounded by the non-stationarity and noisiness of the data. Even though their training loss steadily improves, their validation macro-F1 follows a really noisy trajectory, being rather constant from the epoch 80 onwards. As we will see in the following illustration 3, this isn't so clearly a sign of overfitting but rather that the model isn't able to capture the underlying relationships in the data.

These two observatories shed some light on the possible answer to our thesis: On one hand, the models may not be able to harness the correct symmetries in the data (and forcing time warping symmetry in the case of LSTM and node permutation symmetry in the case of GNNs may be misleading them). On the other hand, the models may actually be able to capture some of the relationships in the data, but the signals present in the data may be too weak and noisy for the model to learn them effectively, which would be in line with the insights of [6]. As additional evidence for this second hypothesis, we present in Figure 3 the final test scores for all models, including the Random Forest. The main thing we would like to highlight is something we can observe in the left-most plot: All models have Validation macro-F1 and Test macro-F1 scores that are very close to each other, which may be a sign that (i) the models aren't overfitting (since they're able to generalize to the test set) but also that (ii) the models aren't able to capture much more than noise in the data, as their performance is really similar to each other and doesn't show a significant improvement over the baselines.

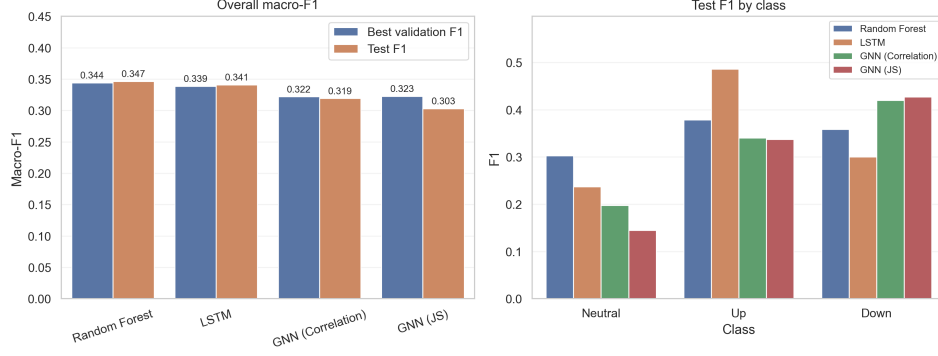


Figure 3: Comparison of all models' performance. Validation and Test macro-F1 on the left, and Test F1 divided by the class distribution on the right.

From the models' performance, we can get several conclusions: (i) graph-based models do not clearly outperform simpler architecture. Both the RF and the LSTM reach slightly higher macro-F1 scores on the test set than the GNNs. Under our experimental setup (whose conclusions may not be directly transferred to other settings, as we will discuss in the next section), the inductive bias introduced by the GNN architecture doesn't seem to be beneficial for forecasting stock movements.

5.2 Qualitative Analysis

In addition to the conclusions that we can derive from the different metrics, we provide, in the following paragraphs, an analysis on the relative performance of the models depending on the class. In order to do so, we include in Figure 4 the Confusion Matrices for the best checkpoints of each model, and we analyze the performance of the models depending on the class, as well as the individual metrics for each model, which we include in Table 1.



Figure 4: Confusion Matrices for the best checkpoints of each model.

Model	Accuracy	Precision Macro	Recall Macro	F1 Macro
Random Forest	0.352	0.349	0.352	0.347
LSTM	0.376	0.345	0.344	0.341
GNN (Correlation)	0.350	0.345	0.338	0.319
GNN (JS)	0.345	0.326	0.328	0.303

Table 1: Test metrics for all models.

The confusion matrices give us a really interesting intuition about the models’ performance. Even though all models have relatively similar overall performance, GNN-based variants are rather effective dealing with the Down class. This comes at the cost of a worse performance on the Neutral class, though. Thus, they seem to have a bias towards predicting more **aggressive** movements. On the other hand, the Random Forest has a more well-balanced approach, having similar errors in all classes (probably, due to the class-balance mechanism inherent in the model training, which forces the model to pay attention to all classes). The LSTM, last, has a behavior that opposite to the GNNs, being more effective at predicting the Up class and having a worse performance on the other two classes. Therefore, it tends to have a bias towards predicting more **positive** movements. These insights suggest that the models are able to capture some of the relationships in the data, but they may not be able to capture all of them effectively, which may be due to the noisiness of the data and the weakness of the signals present in it.

Last of all, these conclusions can also be backed by the individual metrics. GNN-based variants have a slightly higher accuracy than recall, but a significantly lower precision than the other models, which is in line with the fact that they have a bias towards predicting more aggressive movements (which may be correct more often than not, but also leads to more false positives). The Random Forest seems to achieve a good equilibrium between precision and recall.

6 Conclusions and Future Work

Our experiments show that, under our setup, imposing a graph structure over the S&P 100 does not lead to clear improvements in aggregate predictive performance over strong non-graph baselines. While the GNN variants change the pattern of errors across classes, they do not surpass the Random Forest or the LSTM in macro-F1, which suggests that the inductive biases encoded by standard message-passing architectures are not automatically aligned with the structure of financial time series. This observation is consistent with recent work arguing that conventional GNNs may be ill-suited for stock prediction because they do not fully exploit temporal information or future-aware signals, and may even propagate noise through the graph [6].

A key limitation of our study is that we focus on static graphs built from historical correlations and sector information, combined with relatively standard GNN layers. As discussed by Liu et al. [6], more expressive designs that incorporate future-aware distillation, richer temporal modelling, or adaptive graph construction can substantially change the role of graph geometry in stock forecasting. In this sense, our negative result should be interpreted as evidence about a specific family of graph constructions and architectures, rather than as a definitive answer on whether graph structure is beneficial in general. Exploring temporal or event-driven graphs, and architectures explicitly designed for such inputs [7, 12], is a natural next step.

Finally, several recent works suggest that the value of graph-based models may emerge more clearly when they are coupled with complementary information sources. Recent works propose enriching GNNs with news-derived signals extracted by large language models, showing that text-informed edges and node features can improve the prediction of stock movements [2]. Building on this idea, future work could integrate real-time textual data (news, reports, or social media) into our framework, using LLMs to construct dynamic, information-aware graphs on top of the price-based structure. This direction would allow us to test whether graph geometry becomes more beneficial when it encodes both quantitative co-movements and qualitative relationships between firms.

Contribution Statement

While both of us developed the code together and contributed to the writing of the report, there are certain sections that were implemented primarily by each one of us.

- **Antonio Lorenzo Díaz Meco:** Code for the 3 models (Random Forest, LSTM, and GNN), and experimental setup. Writing of the Method and Experimental Setup. Also in charge of the figures included in the report, except for the GNN architecture illustration.
- **Javier Ríos Montes:** Code for data preprocessing and Repository structure. Writing of the Introduction, Related Work, Background, Results and Discussion, and Conclusions and Future Work.

AI Tools Disclosure Statement

In order to complete the project, we used the following AI tools:

- **GitHub Copilot:** Mainly during the data preprocessing stage, to write code for feature engineering and data cleaning. Additionally, it was marginally used throughout the whole codebase to solve minor coding issues and bugs.
- **Perplexity AI:** In our personal experience, Comet is the best one for brain-storming and generating ideas, as well as for improve the legibility of the report. We used it mainly for these purposes, and to a lesser extent for writing assistance, such as recovering useful sources and references for our work.
- **ChatGPT Codex:** Given its ability to analyze the complete structure of a codebase, we used it to review the code and suggest points of failure in the connection between individual components, as well as to suggest improvements in the code structure and modularity.
- **Claude:** While Comet is better for brainstorming and writing, we found that Claude is better for code generation. We used it mainly to, given the pseudocode of an architecture or a function, generate the corresponding code in Python. Additionally, we used it to generate the illustration of the GNN architecture included in the report.

References

- [1] Suresh Chavhan, Praveen Raj, Prayas Raj, Ashit Kumar Dutta, and Joel J.P. C. Rodrigues. Deep learning approaches for stock price prediction: A comparative study of lstm, rnn, and gru models. In *2024 9th International Conference on Smart and Sustainable Technologies (SpliTech)*, pages 01–06, 2024. doi: 10.23919/SpliTech61897.2024.10612666. URL <https://ieeexplore.ieee.org/document/10612666>.
- [2] Zihan Chen, Lei Zheng, Cheng Lu, Jialu Yuan, and Di Zhu. Chatgpt informed graph neural network for stock movement prediction. *SSRN Electronic Journal*, 2023. ISSN 1556-5068. doi: 10.2139/ssrn.4464002. URL <http://dx.doi.org/10.2139/ssrn.4464002>.
- [3] Noelia Crespo González. Predicción de series temporales financieras con modelos de machine learning. Unpublished, 2025. URL <https://oa.upm.es/91012/>.
- [4] Richard A. Davis and Mikkel S. Nielsen. Modeling of time series using random forests: theoretical developments, 2020. URL <https://arxiv.org/abs/2008.02479>.
- [5] Yawei Li, Shuqi Lv, Xinghua Liu, and Qiuyue Zhang. Incorporating transformers and attention networks for stock movement prediction. *Complexity*, 2022:1–10, 02 2022. doi: 10.1155/2022/7739087. URL https://www.researchgate.net/publication/358897376_Incorporating_Transformers_and_Attention_Networks_for_Stock_Movement_Prediction.
- [6] Zhipeng Liu, Peibo Duan, Mingyang Geng, and Bin Zhang. A distillation-based future-aware graph neural network for stock trend prediction. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, page 1–5. IEEE, April 2025. doi: 10.1109/icassp49660.2025.10889901. URL <http://dx.doi.org/10.1109/ICASSP49660.2025.10889901>.

- [7] Antonio Longa, Veronica Lachi, Gabriele Santin, Monica Bianchini, Bruno Lepri, Pietro Lio, Franco Scarselli, and Andrea Passerini. Graph neural networks for temporal graphs: State of the art, open challenges, and opportunities, 2023. URL <https://arxiv.org/abs/2302.01018>.
- [8] Daiki Matsunaga, Toyotaro Suzumura, and Toshihiro Takahashi. Exploring graph neural networks for stock market predictions with rolling window analysis, 2019. URL <https://arxiv.org/abs/1909.10660>.
- [9] Meet Satishbhai Sonani, Atta Badii, and Armin Moin. Stock price prediction using a hybrid lstm-gnn model: Integrating time-series and graph-based analysis, 2025. URL <https://arxiv.org/abs/2502.15813>.
- [10] Koushik Sutradhar, Sourav Sutradhar, Iqbal Ahmed Jhimel, Suneet Kumar Gupta, and Mohammad Monirujjaman Khan. Stock market prediction using recurrent neural network’s lstm architecture. In *2021 IEEE 12th Annual Ubiquitous Computing*, pages 0541–0547, 2021.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- [12] Tianye Wang. Enhancing stock market prediction with temporal graph neural networks and large language model-based explainability. *Procedia Computer Science*, 274:147–160, 2025. ISSN 1877-0509. doi: <https://doi.org/10.1016/j.procs.2025.12.015>. URL <https://www.sciencedirect.com/science/article/pii/S1877050925037366>. 22nd International Multidisciplinary Modeling and Simulation Multiconference (I3M 2025).

A Dataset Details

The dataset consists of daily observations for each of the 88 stocks in the S&P 100 index over the period from January 4, 2021 to December 31, 2025. Each observation includes the following identifiers, features, and labels:

1. Identifiers and Basic Information:

- Ticker symbol
- Date
- Adjusted closing price
- Daily Log Return: $\log\left(\frac{P_t}{P_{t-1}}\right)$, where P_t is the adjusted closing price at time t .
- Future Weekly Log Return: $\sum_{i=1}^5 \log\left(\frac{P_{t+i}}{P_{t+i-1}}\right)$, used only to define the prediction target 5 trading days ahead.
- Class label: a categorical variable indicating the direction of the stock price movement (up, neutral, down), defined in terms of the sign of the future Weekly Log Return.

2. Volume:

- Daily trading volume
- Normalized trading volume: the daily trading volume divided by the 20-day rolling average trading volume of the same stock.

3. Technical Indicators:

- Weekly Moving Average (5MA): the average of the adjusted closing price over the past 5 trading days.
- Monthly Moving Average (20MA): the average of the adjusted closing price over the past 20 trading days.
- Relative Strength Index (RSI): a momentum oscillator that measures the speed and change of price movements, calculated as $RSI = 100 - \frac{100}{1+RS}$, where RS is the average gain of up periods divided by the average loss of down periods over a specified time frame (14 trading days in our case).

- Rolling Volatility: the standard deviation of the daily log returns over a rolling window of 20 trading days.
4. Structural Information:
- Market sector: a categorical variable indicating the market sector to which the stock belongs (e.g., Technology, Healthcare, Financials, etc.).
 - Market Cap: a continuous variable indicating the market capitalization of the company (in dollars).

B Computation of the Graph Structure

Constructing the graphs implied two steps: The first one, common to both graphs, was to connect those stocks belonging to the same market sector. To avoid over-connectivity, we only connect each node to a maximum of 5 neighbors. The second step was to compute the specific relationships between stocks for each graph, which we describe below:

- **Correlation graph:** We first compute the Pearson correlation coefficient between the daily log returns of each pair of stocks using the training set data. This statistic can be expressed, mathematically, as:

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (3)$$

where x_i and y_i are the daily log returns of stocks x and y at time i , and $\bar{x}$ and $\bar{y}$ are their respective means. We then connect each stock to its top- k most correlated neighbors (highest positive magnitude), with k set to 5 in our experiments.

Then, the edge weights are defined as the normalized correlation coefficients, so that they lie in the range $[0, 1]$. Thus, most of them lie in the range $[0.5, 1]$, as we only consider the top-5 most correlated neighbors for each stock.

This relationship is added by means of a double-headed edge, since the GNN uses directed edges but the correlation relationship is symmetric.

- **Jensen-Shannon divergence graph:** This can be understood as going one step further in the same direction as the correlation graph. Instead of summarizing the relationship between two given stocks purely by the point-wise correlation between their returns, we can compare the entire distribution of returns for each stock and connect those stocks with similar return distributions.

To do so, we first compute the daily log returns for each stock and create a histogram of these returns (with 30 bins). All histograms use the same bin edges, defined by the minimum and maximum log returns across all stocks in the training set. After representing these bins, we normalize each one so that they can be interpreted as probability distributions.

Then, we compute the Jensen-Shannon divergence between the return distributions of each pair of stocks, which can be expressed as:

$$JSD(P||Q) = \frac{1}{2}D_{KL}(P||M) + \frac{1}{2}D_{KL}(Q||M) \quad (4)$$

where P and Q are the return distributions of two stocks, $M = \frac{1}{2}(P + Q)$ is the average distribution, and D_{KL} is the Kullback-Leibler divergence. After computing this for every pair of stocks, we connect each individual stock to its top- k closest neighbors (lowest divergence), with k set to 5 in our experiments. The edge weights are defined as the normalized inverse of the Jensen-Shannon divergence, so that they lie in the range $[0, 1]$, with higher values indicating more similar return distributions (less distance between them).

C Important Details

C.1 Sector Information

The 88 stocks that we are considering in our dataset belong to the following market sectors:

- Financial Services: 16 stocks
- Healthcare: 14 stocks
- Technology: 12 stocks
- Industrials: 12 stocks
- Consumer Defensive: 10 stocks
- Consumer Cyclical: 9 stocks
- Communication Services: 7 stocks
- Energy: 3 stocks
- Utilities: 3 stocks
- Real Estate: 2 stocks

C.2 Class Distribution

The distribution of the class labels (up, neutral, down), given $\epsilon = 0.01$, is as follows:

- Up: 41.3%
- Neutral: 24.85%
- Down: 33.85%

This distribution is relatively balanced and, moreover, may reflect a certain intuition about the stock market, in which, over a week, stocks are more likely to go up or down than to remain neutral (as small fluctuations are not labeled as significant movements) and, overall, there is a slightly higher probability of stocks going up than down (which may be due to the general growth trend of the stock market over time). This distribution should allow the models to learn effectively without being biased towards a particular class.